



An efficient network-based formulation for sequence dependent setup scheduling on parallel identical machines

Bradley E. Anderson^{a,*}, James D. Blocher^b, Kurt M. Bretthauer^b,
Munirpallam A. Venkataramanan^b

^a Department of Operational Sciences, Air Force Institute of Technology, Wright-Patterson AFB, OH 45433, United States

^b Department of Operations and Decision Technologies, Kelley School of Business, Indiana University, Bloomington, IN 47405, United States

ARTICLE INFO

Article history:

Received 18 May 2009

Received in revised form 23 January 2012

Accepted 18 June 2012

Keywords:

Sequence-dependent setups

Scheduling

Mixed integer programming

Earliness

Tardiness

ABSTRACT

This paper compares the efficacy of a newly developed network-based mixed-integer programming (MIP) formulation with three existing formulations for the sequence dependent setup scheduling problem with earliness/tardiness penalties. This research shows that the new model is more efficient in terms of computation time for larger multi-machine problems than the existing formulations of these problems. The mixed-integer nature of the formulation allows companies to solve this class of problems with any one of many commonly available integer programming software packages. The presented MIP formulation provides a unique and useful method of conceptualizing and modeling a practical, yet difficult, problem within industry.

Published by Elsevier Ltd

1. Introduction

Developing production schedules is an important task manufacturing firms face every day, as consistently generating efficient schedules can result in substantial improvements in productivity. This problem is made more difficult when producing multiple products on multiple limited-capacity resources, where each product produced has sequence-dependent setup times. Panwalker et al. [1] found, in a survey of industrial schedulers, that 70% of the schedulers reported at least some operations they schedule are subject to sequence dependent setup times. Thirteen percent of the schedulers reported that all the operations they schedule are subject to sequence dependent setups. With today's emphasis on Just-in-Time manufacturing, minimizing earliness and tardiness are even more important scheduling objectives. Panwalker et al. also reported that industrial schedulers overwhelmingly named meeting due dates as their most important scheduling criterion.

Setups are sequence dependent when the time to setup for a product x depends on which product was setup on the machine prior to running product x . This problem naturally arises when a machine is producing parts with the same shape, but different colors. A small amount of residual white pigment has little impact on the shade of black; however, even a small amount of black pigment can notably change the shade of white. Thus, the equipment cleaning time between runs of one color and another depend on the colors themselves. Less cleaning time is required when the machine changes from white to black, as opposed to from black to white.

Another situation where sequence dependent times are common occurs when a resource produces two products (or more) and each product has two (or more) variations. If the next product to be produced is the second variation of product A , the time it takes to change the machine from the first variation of product A would generally be much lower than if the machine were changed from one of the variations of the other product B .

* Corresponding author. Tel.: +1 937 255 3636x4646; fax: +1 937 656 4943.

E-mail addresses: bradley_anderson@sbcglobal.net, bradley.anderson@afit.edu (B.E. Anderson).

The problem addressed here is one of scheduling N jobs on M identical parallel machines in a deterministic environment. Each job has associated with it a processing time p_i and a due date d_i . There is a sequence dependent setup time s_{ij} associated with each pair of jobs representing the time to setup for job j after job i was processed previously. Only one job can be processed at a time on a given machine and we assume for convenience that all jobs are available at time 0, although the formulation could easily be modified to include ready times. The completion time of each job is denoted c_i so that $t_i = \text{Max}\{c_i - d_i; 0\}$ is the tardiness for job i and $e_i = \text{Max}\{d_i - c_i; 0\}$ is the earliness for job i . The objective is to minimize total earliness and tardiness, $\sum(e_i + t_i)$, over all jobs, $i = 1 \dots N$. According to the scheduling need, this objective could easily be just earliness, just tardiness, or some weighted combination of the two.

A review of the literature shows very few published articles address scheduling jobs to minimize total earliness and tardiness (ET) with sequence-dependent setup times. Almost all ET articles assume that setup time is either negligible or sequence-independent. Sequence-dependent setup times are very important to some industries, and Baker [2] states a sequence-dependent setup time is a defining characteristic of classical job-shop scheduling.

Zhu and Heady [3] addressed this problem most recently by presenting two multi-machine MIP formulations. One general formulation uses a triple index notation, and one uses a double index formulation that requires reasonable setup times, but is more efficient in terms of the number of variables and constraints needed. A similar multi-machine research effort on this problem was presented by Balakrishnan et al. [4]. The Balakrishnan et al. formulation requires setup times for sequential jobs on any machine satisfy the triangular law of inequality. The model proposed here and the models by Zhu and Heady do not require this assumption. The single machine MIP version of the problem developed by Coleman [5] was a predecessor to these multi-machine formulations. Chen [6] utilized a dynamic programming approach, but addressed only two types/batches of jobs on a single machine. Although they did not develop an optimal solution procedure, Sivrikaya-Serifoglu and Ulusoy [7] employed a genetic heuristic method to solve the multiple-machine problem we study here.

Zhu and Wilhelm [8] presented a good literature review of scheduling and lot-sizing with sequence-dependent setups. Their literature review of sequence-dependent setup research validates the usefulness of this research. They point out that the most practical scheduling problems involve setup times or costs. They go on to say, "Because of their prevalence in, and importance to, industry and because of the challenges they present to solution methodologies, scheduling problems that involve a Sequence-Dependent Setup (SDS) have attracted attention from many researchers". In their summary of research they identify only the Balakrishnan et al. paper, used as the primary point of comparison for this research, to have presented an MIP based parallel machine sequence-dependent ET solution. The Zhu and Wilhelm paper specifically points out that future research with optimization techniques could bridge a knowledge gap. A current literature review has revealed no other recently presented optimization methods for this particular problem.

Thus, in this paper we propose a new MIP formulation for solving the sequence-dependent setup-scheduling problem on multiple parallel machines for the earliness/tardiness objective. The formulation is based on the concept of network flows. We compare the new network formulation to that of Zhu and Heady and to that of Balakrishnan et al.. Using the CPLEX math programming package [9], we show computationally that as the number of machines increases, the performance gap widens in favor of our new network formulation.

2. Formulations

In this section, a new formulation is presented for the sequence dependent setup scheduling problem discussed earlier. In addition, the three other formulations for the computational comparison are also presented and discussed.

2.1. Network formulation

The formulation is based on the traveling salesman formulation and network flow concepts. The traveling salesman (TSP) part of the formulation ensures a sequence of jobs. The network flow portion, where flow represents time, is used to (1) eliminate subtours, and (2) allow the introduction of due dates to determine tardiness. The basic TSP formulation can be stated as a problem in which N cities must be visited by traveling a minimum distance by a single traveling salesman. The distance associated with traveling between cities i and j is denoted s_{ij} . The objective function serves to minimize the sum of the travel distances between each of the locations.

This formulation can be viewed as an adaptation of a model to solve the Traveling Salesman Problem (TSP) where the sub-tour elimination constraints and the mechanism to account for the elapsed time are integrated through two different flow principles.

In the TSP, each city is visited sequentially by a salesman in one continuous route (i.e., no sub-tours). In the single machine scheduling problem (SMSP), each job must be completed sequentially on a machine. In the TSP, there is transportation time to get from one city to the next and a service time at each city, while in the SMSP, there is a changeover time to get from one job to another and a processing time for each job.

The formulation will be illustrated with a simple example problem having three jobs with the processing times and changeover times given (See Tables 1 and 2).

Total processing time = $10 + 12 + 15 = 37$.

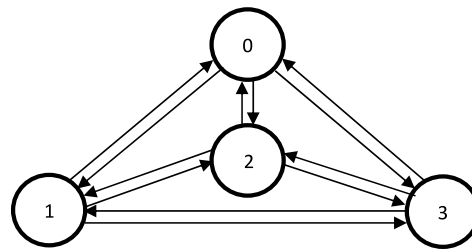
Highest possible changeover time = $10 + 15 + 15 = 40$.

Table 1
Processing time.

Job	1	2	3
Processing time	10	12	15

Table 2
Changeover times.

From job	To job		
	1	2	3
1	–	10	8
2	10	–	15
3	8	15	–

**Fig. 1.** Four-node network.

In this example there are three jobs to be scheduled on one machine, and the example can be represented as a simple four-node network (See Fig. 1).

In order to make sure each jobs is processed, and processed only once, and that both processing time and changeover time is accounted for, two types of connections between each node are needed, one for path and one for flow. The path connections (the x_{ij} variables in the model) can be viewed as on and off switches establishing the path infrastructure connections and the correct directions the flow will take. The second type of connection is flow, where the flow is the amount of time needed for the changeover and processing time (the y_{ij} variables in the model). The flow is shipped over the path infrastructure connections.

The model allows only one “path” connection into and out of each job ensuring that jobs are scheduled only once. The model also has a source node (node 0) that has more than a sufficient quantity of time (B) to be “shipped” over the established path connections, thus accounting for the changeover and processing time of all the jobs. The model restricts the shipments (represented by the y_{ij}) over only those paths which have been established (those paths with $x_{ij} = 1$), thus making sure that the flow follows a valid path.

Each node (each job) is required to consume a certain amount of flow (process plus changeover time), and this combination of the path and flow variables gives the network (job schedule) unique properties. The concept of separating the pathway connections from time flow through two sets of variables allows this model to be easily adapted for various objective functions of scheduling problems, multiple machines, and sequence dependent setup times.

In the example problem, the source node 0 should have a minimum flow of $B = 77$ to account for the maximum total possible processing and changeover time (maximum of 37 processing and 40 changeover). Note that any $B \geq 77$ will accurately model the example problem, since any excess “time” which flows through the network will be leftover.

In the example, if node 0 is connected to node 3, $x_{03} = 1$ so that 77 units are shipped over y_{03} . A total of 15 units of time will be consumed by node 3 (job 3 processing time is 15 and the initial changeover time is assumed to be 0). Next the set of constraints representing only one outlet from node 3 and flow requirements in node 1 and 2 not being met will ensure node 3 will be connected to node 2 or 1. Assume that node 1 (job 1) is next, $x_{31} = 1$, so that $y_{31} = 77 - 15 = 62$. A total of 18 units of time will be consumed by node 1 (job 1 requires 10 for processing plus 8 for changeover). Finally, $x_{12} = 1$ so that node 2 (job 2) finishes the schedule. Thus, $y_{12} = 62 - 18 = 44$ and node 2 consumes 22 units of time (12 for processing plus 10 for changeover). Because the schedule is now complete, node 2 will connect to node 0 with $x_{20} = 1$ and $y_{20} = 22$. Based on job 3 as the starting job and job 2 as the final job, returning unused time in original $B(77)$ back to the source will be $44 - 22 = 22$ units. In the example problem the setup times are given as symmetric while the model can accommodate asymmetric changeover times. The model can accommodate an initial setup time, as well as a change over time from the last job to first job in a sequence, if the problem calls for one.

The completion time of the jobs, the difference between the job completion times in a schedule, the job due dates, the amount of changeover time, and finally the total objective function value, can all be deduced from the combination of the x_{ij} variables that are equal to 1 and the y_{ij} variables that are greater than 0. This formulation can easily be modified to handle m multiple identical machines as long as there are at most m flows out of the source node 0.

Now that we have described the foundations of our network formulation in terms of the Traveling Salesman Problem, the TSP formulation can then be transformed into a Due Date TSP (DDTSP). Each city is to be visited by its due date and the objective function is then one of minimizing the tardiness, versus minimizing travel distance. The constant B is introduced and must be sufficiently large. The constant B can be viewed as a bucket of time that needs to contain enough total time to allow all the setup and job times to be completed.

The DDTSP can then be transformed into a single machine ET scheduling problem with sequence-dependent setup times. The DDTSP is very similar to a single machine scheduling formulation with sequence dependent changeover times. Each job to be scheduled is analogous to a city that must be visited by the due date. If s_{ij} is defined as the setup time between job i to job j (instead of the travel time between city i and city j), the only changes to the DDTSP that must be made is to include the processing time of the job, an earliness variable in the objective function, and a constraint to take the appropriate amount of time from the bucket of time (B). The problem is now one of minimizing the earliness and tardiness of completed jobs, versus minimizing the tardiness of visiting cities. The decision variable y_{ij} continues to represent the flow from node i to node j . However, due to the introduction of job processing times, this flow now represents the time remaining until completion of all scheduled jobs and decreases by the total of the processing and setup time of each job in the sequence. This allows us to obtain the earliness and tardiness of each job scheduled in the sequence and minimize it in the objective function. This problem can be referred to as the $N/1/ET$ with sequence-dependent setups, where N refers to the number of jobs and 1 indicates one machine.

Finally, this formulation is modified to allow weighting of the earliness and tardiness objectives, multiple machines, and release dates. Utilizing the network structure of the $N/1/ET$ with sequence-dependent setups model, it is now a simple matter to encompass the added complexity of multiple identical machines. Adding release dates to the formulation and weighting the earliness and tardiness penalties are also simple tasks. Constraints are slightly modified by separating out the assignment constraints into and out of the imaginary starting job 0. These assignment constraints allow a number of imaginary starting jobs equal to or less than the number of machines available. This problem can be referred to as the $N/M/ET$ with sequence-dependent setups and release dates, or $N/M/ET$ -SR for short.

To present the new network flow formulation, some additional notation is needed beyond that presented in the last section. Let B be a large constant, x_{ij} be a binary decision variable equal to 1 if job j immediately follows job i , 0 otherwise, and y_{ij} be non-negative continuous decision variables. The formulation is then:

$$\text{Minimize } \sum_{i=1}^N (t_i + e_i) \quad (1)$$

$$\text{Subject to } \sum_{\substack{j=0 \\ j \neq i}}^N x_{ij} = 1 \quad \forall i = 1 \dots N \quad (2)$$

$$\sum_{\substack{i=0 \\ i \neq j}}^N x_{ij} = 1 \quad \forall j = 1 \dots N \quad (3)$$

$$\sum_{j=1}^N x_{0j} \leq M \quad (4)$$

$$\sum_{i=1}^N x_{i0} \leq M \quad (5)$$

$$y_{0j} = B \cdot x_{0j} \quad \forall j = 1 \dots N \quad (6)$$

$$\sum_{\substack{i=0 \\ i \neq j}}^N y_{ij} - \sum_{\substack{i=0 \\ i \neq j}}^N y_{ji} \geq p_j + \sum_{\substack{i=0 \\ i \neq j}}^N (x_{ij} \cdot s_{ij}) \quad \forall j = 1 \dots N \quad (7)$$

$$y_{ij} \leq B \cdot x_{ij} \quad \forall i = 1 \dots N, \quad \forall j = 0 \dots N, \quad i \neq j \quad (8)$$

$$B - \sum_{\substack{j=0 \\ j \neq i}}^N y_{ij} + e_i = d_i + t_i \quad \forall i = 1 \dots N \quad (9)$$

$$x_{ij} = 0 \text{ or } 1; \quad y_{ij}, e_i, t_i \geq 0. \quad (10)$$

Eq. (1) is the objective function that minimizes the sum of the earliness and tardiness. The first two constraints (2) and (3) ensure that each job appears once and only once in a schedule. The binary x_{ij} decision variables are equal to one when job j immediately follows job i in a sequence and are equal to zero otherwise. The index of zero on the variable x_{0j} indicates the starting point for the schedule so that job j is first in the sequence. Similarly, x_{j0} is a variable which indicates the end of

the schedule. Constraints (4) and (5) allow at most M machines to be scheduled: the inequality is needed primarily for cases when $N < M$, but for most problems, equality will hold.

The next three constraints, (6)–(8) are flow constraints, where the unit of flow in this case is *time*. The decision variable y_{ij} represents the flow of time from node i to node j (job i to job j). Constraint (6) establishes an initial flow of B time units for each machine sequence, where there are at most M sequences. B must be large enough to cover changeover, processing, and inserted idle time. Constraint (7) ensures that time is “dropped” at a node as the node is added to the sequence. The difference in the flows in and out of a node must be sufficient to cover the changeover and processing time required by the job represented by the node. The inequality allows for inserted idle time, but this idle time could be explicitly modeled by adding another variable to constraint (7) and making the constraint an equality constraint. Constraint (8) ties the sequence decision variables to the flow decision variables so that flow is allowed between i and j only if job i is sequenced before j .

Constraint (9) is a balance constraint used to determine the earliness or tardiness for each job in each machine's sequence based on the flows and the due dates. Finally, constraint (10) defines the 0, 1 and non-negative terms.

2.2. Zhu and Heady formulation

The Zhu and Heady triple index formulation utilizes many of the same parameters and variables as the network formulation. A continuous variable X_i is used to indicate the completion time of job i while the binary decision variables Y_{ijm} and Z_{im} are used to indicate, respectively, if job i immediately precedes job j on machine m , and, if job i is scheduled on machine m .

$$\text{Minimize } \sum_{i=1}^N (t_i + e_i) \quad (11)$$

$$\text{Subject to } X_i - t_i + e_i = d_i \quad \forall i = 1 \dots N \quad (12)$$

$$\sum_{m=1}^M Z_{im} = 1 \quad \forall i = 1 \dots N \quad (13)$$

$$\sum_{\substack{j=1 \\ i \neq j}}^N Y_{ijm} \leq Z_{im} \quad \forall i = 0 \dots N, \forall m = 1 \dots M \quad (14)$$

$$\sum_{\substack{i=0 \\ i \neq j}}^N Y_{ijm} = Z_{jm} \quad \forall j = 1 \dots N, \forall m = 1 \dots M \quad (15)$$

$$X_i - X_j - BY_{ijm} \geq p_i + s_{ji} - B \quad \forall i = 1 \dots N, \forall j = 0 \dots N, i \neq j, \forall m = 1 \dots M \quad (16)$$

$$Y_{ijm} Z_{im} = 0 \text{ or } 1; \quad X_i, e_i, t_i \geq 0. \quad (17)$$

The objective function (11) minimizes the sum of the earliness and tardiness. The first constraint (12) measures the degree to which each job is tardy or early. The second constraint (13) ensures each job is processed on one and only one machine. The next constraints (14) and (15) ensure that each job comes immediately before and immediately after only one other job. Constraint (16) ensures the completion time of job i is far enough after job j to include the processing time and setup time of job i . The last constraint (17) defines the 0, 1 and non-negative terms.

The Zhu and Heady double index formulation takes advantage of certain symmetries inherent in the problem, and reduces the number of binary variables by the difference between $N(N+1)M$ and $N(N-1)/2$. A new zero-one variable, Y_{ij} , replaces Y_{ijm} where $Y_{ij} = 1$, if job i precedes job j , and is 0 otherwise. Y_{ij} does not refer to the job's machine, and does not require that i be immediately before j . This formulation requires that setup values not exceed $p_{km} + s_{ik} + s_{kj}$. By substituting Y_{ij} and noting the relationship $Y_{ij} + Y_{ji} = 1$, constraints (13) and (14) can be eliminated and constraint (15) can be replaced with two new constraints.

$$\text{Minimize } \sum_{i=1}^N (t_i + e_i) \quad (18)$$

$$\text{Subject to } X_i - t_i + e_i = d_i \quad \forall i = 1 \dots N \quad (19)$$

$$\sum_{m=1}^M Z_{im} = 1 \quad \forall i = 1 \dots N \quad (20)$$

$$X_j - X_i - BY_{ij} - BZ_{im} - BZ_{jm} \geq p_{jm} + s_{ij} - 3B \quad j \neq i, \forall i = 0 \dots N, \forall j = i + 1 \dots N, \forall m = 1 \dots M \quad (21)$$

$$X_i - X_j + BY_{ij} - BZ_{im} - BZ_{jm} \geq p_{im} + s_{ji} - 2B \quad j \neq i, \forall i = 1 \dots N, \forall j = i + 1 \dots N, \forall m = 1 \dots M \quad (22)$$

$$\sum_{m=1}^M Z_{0m} = M \quad (23)$$

$$\sum_{j=1}^N Y_{0j} = N \quad (24)$$

$$Y_{ij}, Z_{im} = 0 \text{ or } 1; \quad X_i, e_i, t_i \geq 0. \quad (25)$$

Constraints (19) and (20) are identical to constraints (12) and (13). Constraints (21) and (22) replace constraint (16) and represent job precedence relationships. These two constraints have modified ranges for index i from the version originally published by Zhu and Heady [3] and presented in Computers & Industrial Engineering. Also, it was found that constraints (23) and (24) were needed. Constraint (25) is the same as constraint (17), except that the notation is changed to double subscripts.

2.3. Balakrishnan et al. formulation

To present the Balakrishnan et al. formulation, one new variable is needed. The variable X_{ij} is used to indicate if job i precedes job j on the same machine. (Note that job i does not necessarily immediately precede job j . Also note that this capital X_{ij} is not the same as the previously used lower case x_{ij} .) The other parameters and variables are the same as defined for the previous two formulations.

$$\text{Minimize } \sum_{i=1}^N (t_i + e_i) \quad (26)$$

$$\text{Subject to } \sum_{m=1}^M Z_{im} = 1 \quad \forall i = 1 \dots N \quad (27)$$

$$Z_{im} + \sum_{i \neq j}^N Z_{jm} + X_{ij} \leq 2 \quad \forall i = 0 \dots N, m = 1 \dots M \quad (28)$$

$$c_i + e_i - t_i = d_i \quad \forall i = 1 \dots N \quad (29)$$

$$c_j - c_i + B(3 - X_{ij} - Z_{im} - Z_{jm}) \geq p_j + s_{ij} \quad \forall i = 1 \dots N - 1, \forall j = i + 1 \dots N, \forall m = 1 \dots M \quad (30)$$

$$c_i - c_j + B(2 - X_{ij} - Z_{im} - Z_{jm}) \geq p_i + s_{ji} \quad \forall i = 1 \dots N - 1, \forall j = i + 1 \dots N, \forall m = 1 \dots M \quad (31)$$

$$c_i \geq s_{0i}Z_{im} + p_iZ_{im} \quad \forall i = 1 \dots N, \forall m = 1 \dots M \quad (32)$$

$$X_{ij}, Z_{im} = 0 \text{ or } 1; \quad c_i, e_i, t_i \geq 0. \quad (33)$$

The objective function (26) minimizes the sum of the earliness and tardiness. The first constraint (27) assigns each job to exactly one machine. The second constraint (28) ensures that the issue of job precedence between jobs i and j is relevant only if both jobs are assigned to the same machine. Constraint (29) calculates the earliness or tardiness in job i based on its completion time. Constraints (30) and (31) establish the relationship between the completion times of jobs i and j as long as both jobs are assigned to the same machine. These constraints enforce that there is sufficient time between the completion of jobs i and j , based on the order of job precedence. Constraint (32) is used to start each sequence for each machine, so that a setup time can be included from the state of the machine at time 0 to the first job in the sequence. Finally, constraint (33) defines the 0, 1 and non-negative terms. Table 3 is a summary of the notation used in the formulations discussed.

3. Computational results

A computational study was carried out to compare the performance of the new network formulation to the formulations of Zhu and Heady and Balakrishnan et al.. Two different studies were performed, one with setup times satisfying the triangular inequality restriction and one with general setup times. Only the first study with the triangular inequality restriction is presented here, since the triangular inequality restriction typically is satisfied in practice. In addition, the results of the two studies show no significant differences.

CPLEX version 7.1 was used to solve the mixed-integer formulations and was run on a one giga-hertz personal computer. CPLEX is a commercially available software package that solves mixed-integer problems. CPLEX default settings were used in solving all mixed integer problems so that all cuts were selected automatically and the “best bound” variable selection strategy was utilized.

Five problems were randomly generated for each combination of the four different job sizes (10, 12, 15, and 20 jobs) and four different resource pools (1, 3, 7, and 10 machines) for the network, Balakrishnan et al., and double index Zhu and Heady formulations resulting in a total of 16 experimental cells for both models. Of these 16 experimental cells only the 10

Table 3

Formulation variables.

Network flow formulation variables	
N	Number of jobs to be scheduled
M	Number of machines to be scheduled
t_i	Tardiness of job i
e_i	Earliness of job i
x_{ij}	Binary decision variables equal to one when job j immediately follows job i in a sequence and equal to zero otherwise
y_{ij}	Non-negative continuous decision variables representing the flow of time from node i to node j (job i to job j)
x_{0j}	The starting point of the schedule so that job j is first in the sequence
x_{j0}	The ending point of the schedule so that job j is last in the sequence
p_j	Processing time for job j
s_{ij}	Setup time for job j when it follows job i
d_i	Due date for job i
B	A large constant that represents the total volume of time available to be allocated to each machine sequence; It must be large enough to cover changeover, processing, and inserted idle time.
Zhu and Heady formulations variables ($N, M, t_i, e_i, d_i, s_{ij}, B$ are as listed above)	
X_i	A continuous variable used to indicate the completion time of job i
Y_{ijm}	A binary decision variable used to indicate if job i immediately precedes job j on machine m
Z_{im}	A binary decision variable used to indicate if job i is scheduled on machine m
Y_{ij}	Replaces Y_{ijm} where $Y_{ij} = 1$, if job i precedes job j , and is 0 otherwise. It does not refer to the job's machine, and does not require that i be immediately before j .
p_{km}	Processing time of job k on machine m .
Balakrishnan et al. formulation variables ($N, M, t_i, e_i, d_i, s_{ij}, p_j, B, Z_{im}$ are as listed above)	
X_{ij}	A binary decision variable used to indicate if job i precedes job j on the <i>same</i> machine. (Note that job i does not necessarily immediately precede job j)
c_i	The completion time of job i .

machine problems were not examined for the Zhu and Heady triple index formulation due to excessive computation time. For this comparison, problem parameters on a scale similar to the Balakrishnan et al. research were used. Job processing times were randomly selected from a discrete uniform distribution with a range of 1 to 100 time units. Setup times were randomly selected from a discrete uniform distribution with a range of 10 to 35 time units. The due dates were randomly generated from a discrete uniform distribution with a range of 1 to K time units, where $K = 100 \times N/M$ and thus was different for each experimental cell to maintain the same general degree of due date tightness.

Each problem instance was run for a maximum of two hours (7200 s). If the problem could not be solved in this amount of time, the best solution at the end of the allotted time was recorded for comparison and the “solution time” was recorded as 7200 s. Thus, the reported average solution time in an experimental cell would contain a value of 7200 s if all five problems in the cell were not completely solved.

3.1. Results

We generally found that the new network formulation performed best when there were more machines and the Balakrishnan et al. formulation (BKS) performed best when there were few machines. The Zhu and Heady formulations (denoted ZH double and ZH triple), especially ZH double, performed better as the number of jobs increased on a few machines. Although the double subscripted Zhu and Heady formulation performed better than the triple subscript Zhu and Heady formulation in most instances, it sometimes did poorer for many machines.

In Table 4, for each job–machine–formulation combination, the average amount of time to solve the five problems is on the left in each cell, the percentage solved optimally within the two-hour time limit is given in the middle column of each cell, and the average percent solution gap is on the right.

It can be seen from this computational study that the new formulation does quite well for many machines, and is as good as or better than any of the other formulations for the 7 and 10 machine experimental cells. As the number of machines is reduced to around 3 or 4, the Balakrishnan et al. formulation performs better. Bold cells in Table 4 show the best performing results, for all problems solved for a given number of machines.

One notices some trends from this data. As might be expected, as the number of jobs increases, all of the formulations generally take longer to solve the problems. But notice that even though the problem difficulty generally increases for Balakrishnan et al. and Zhu and Heady when the number of machines increases, this is not the case for the network formulation. In general, as more machines are added in the network formulation, the time to solve problems decreases for any given number of jobs. This issue is discussed in more depth later in this section. It would seem that the difficulty of the problem is a function of something on the order of N/M rather than $N \times M$ as for the other two formulations.

The computational time performance trend is shown in Fig. 2. The pair of dashed lines represents the performance of the Network and Balakrishnan et al. models for the case of 10 jobs while the pair of solid lines represents the performance of the two models for the case of 15 jobs. The *approximate* threshold points where the dominating performance region changes

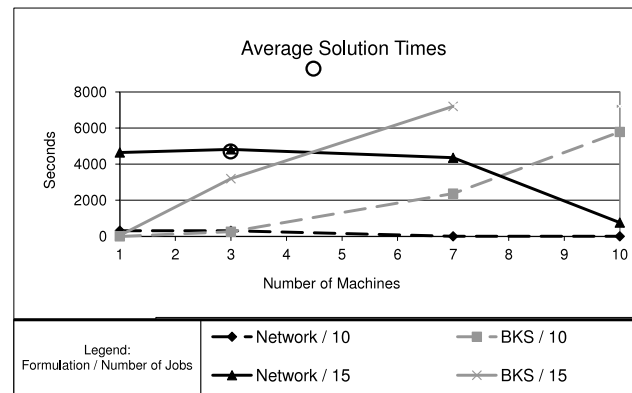


Fig. 2. Graph of average solution times for 10 and 15 job problem sizes.

Table 4

Average run time (in s), percentage of problems solved, and average solution gap.

	Jobs	1 Machine			3 Machines			7 Machines			10 Machines		
		s	% Solved	% Gap	s	% Solved	% Gap	s	% Solved	% Gap	s	% Solved	% Gap
Network formulation	10	314	100	0	307	100	0	2	100	0	1	100	0
	12	393	100	0	1132	80	4	639	100	0	3	100	0
	15	4647	40	39	4820	40	24	4355	60	3	764	100	0
	20	7200	0	100	7200	0	74	6873	20	21	7200	0	31
BKS formulation	10	0.37	100	0	257	100	0	2368	80	1	5783	20	6
	12	0.41	100	0	252	100	0	7200	0	41	7200	0	26
	15	53	100	0	3195	60	7	7200	0	60	7200	0	68
	20	27	100	0	5799	20	31	5774	20	54	7200	0	77
ZH-double formulation	10	0.74	100	0	1535	80	9	4331	40	5	5781	20	6
	12	0.42	100	0	2953	60	12	7200	0	52	7200	0	21
	15	44	100	0	5018	40	28	7200	0	58	7200	0	63
	20	25	100	0	5805	20	48	5762	20	50	7200	0	75
ZH-triple formulation	10	12	100	0	1495	80	1	3411	60	10	7200	0	N/A
	12	6	100	0	2098	80	<1	4816	40	29	7200	0	N/A
	15	3104	60	24	4731	40	35	7200	0	94	7200	0	N/A
	20	5768	20	38	6202	20	72	7200	0	93	7200	0	N/A

Table 5

Number of variables and constraints.

Formulation	Total variables	Binary variables	Constraints
Network	$2N^2 + 4N$	$N^2 + N$	$N^2 + 5N + 3$
BKS	$N(N-1)/2 + MN + 3$	$N(N-1)/2$	$N + 2MN + 3MN(N-1)/2$
ZH (double)	$N(N+1)/2 + (N+1)M + 3N$	$N(N+1)/2 + (N+1)M$	$2(N+1) + 2M(N-1)N/2 + MN$
ZH (triple)	$M(N^2 + N + 1) + 3N$	$M(N^2 + N + 1)$	$M(N^2 + 2N + 1) + 2N$

from one model to the other can be seen as the intersection (circled) of the pair of lines of the different model plots for both the 10 and 15 job problem sizes.

As mentioned earlier, when a problem could not be solved in the allotted two-hour time limit, the objective function was recorded and compared with the lower bound. The ‘bound gap’ shown in Table 4 is computed as the difference between the two bounds divided by the upper bound and then multiplied by 100 for representation as a percent. The Balakrishnan et al. formulation once again showed a performance edge in solution quality (bound gaps) for the 1 and 3 machine problems, the network model again showed a performance advantage in the solution bound gaps achieved in the 7 and 10 machine problems for all job sizes.

3.2. Variable and constraint comparison

It is reasonable to conclude that the solution time advantage of the models is at least partially due to the number of variables and constraints employed by each formulation. Tables 5 and 6 show the number of variables and constraints for each formulation, including a breakout of the number of binary variables in Table 5. Table 6 has a numerical representation of these numbers for each of the problems solved in the computational study.

Table 6

Total number of variables (Var) and constraints (Con) for test problems.

Formulation	Jobs	1 Machine		3 Machines		7 Machines		10 Machines	
		Var	Con	Var	Con	Var	Con	Var	Con
Network	10	240	153	240	153	240	153	240	153
	12	336	207	336	207	336	207	336	207
	15	510	303	510	303	510	303	510	303
	20	880	503	880	503	880	503	880	503
BKS	10	85	165	105	475	145	1095	175	1560
	12	114	234	138	678	186	1566	222	2232
	15	165	360	195	1050	255	2430	300	3465
	20	270	630	310	1850	390	4290	450	6120
ZH (double)	10	96	122	118	322	162	722	195	1022
	12	127	170	153	458	205	1034	244	1466
	15	181	257	213	707	277	1607	325	2282
	20	291	442	333	1242	417	2842	480	4042
ZH (triple)	10	141	141	363	383	807	867	1140	1230
	12	193	193	507	531	1135	1207	1606	1714
	15	286	286	768	798	1732	1822	2455	2590
	20	481	481	1323	1363	3007	3127	4270	4450

From these tables, one can see some possible reasons each of the formulations perform as they do. When some combination of the number of variables and the number of constraints are factored in, the performance results of Table 4 and the Variable/Constraint makeup seems to be correlated.

It is interesting to note that the same number of variables and the same number of constraints are needed for the new network formulation no matter how many machines are used in the problems. In addition, for the multiple-machine cases of this experimental design, the network formulation always requires fewer constraints than the other two formulations. While the Balakrishnan et al. model will always have the fewest binary variables and has the fewest total variables in this experimental design, the number of variables and constraints increases as more jobs and machines are added. Evidently at some point, this increasing number of constraints decreases the performance of the formulation relative to the network formulation where the numbers remain constant. This constraint advantage may explain the network formulation computational advantage for larger problem sizes.

3.3. Additional computational experience

Three additional experimental variables were explored as side experiments of this research. This was done to understand if there were performance differences between the formulations other than that attributed to the number of jobs and machines. These experimental factors were due date tightness, objective function weights on earliness and tardiness, and the relative size of setup times when compared to job processing times.

1. *Due date tightness*: The due date tightness of the study problems was loosened to see the effect on the solution times of the three formulations. This was done by using a larger range from which to pick the due dates. Due dates were randomly generated as before but with $K = 300 \times N/M$. Only the experimental cases of 10, 15, and 20 jobs across 3 and 10 machines were examined. One might expect using relatively loose due dates would reduce the time it takes to solve any particular problem and thus increase the number of problems the formulations could solve in the time allowed. This was indeed what was observed. Table 7 shows the impact on the various formulations. Shaded regions show the greatest change and although no improvement was seen due to loosened due dates for 3 machines with the network formulation, every other category showed improvement where not all jobs had previously been solved. The network formulation showed the largest improvement for the most complex problem set of 20 jobs and 10 machines. This is consistent with the primary experimental results previously shown.
2. *Objective function weights*: The primary experimental design utilized equal weights (equal to one) for the two objective function terms. For this side experiment, unequal weights were used to see if they added discrimination between the three formulations. Random weights were assigned with earliness term weights being chosen from a uniform distribution between 1 and 4, and tardiness term weights derived from a uniform distribution between 1 and 6. Earliness weights were constrained to be less than or equal to tardiness weights in order to give a slightly greater penalty to tardiness. For the experimental cells of 10 and 15 jobs and 3 and 10 machines (the only ones tested), adding the random weights had little or no effect on the formulations' capability to solve the problems. Of the 14 comparison cells, only 5 cells showed any change in the number of problems solved with 4 having a one problem change and 1 cell having a 2 problem change. These changes were across all three formulations, but only for the 3 machine problems. The only interesting aspect is that these small changes were all improvements for problems with weights added.

Table 7

Due date tightness results.

Percent problems solved (larger is better)		100% due date range	300% due date range	100% due date range	300% due date range
Formulation	Problem size (Jobs)	3 Machines (%)	3 Machines (%)	10 Machines (%)	10 Machines (%)
Network	10	100	100	100	100
	15	40	40	100	100
	20	0	0	0	100
Balakrishnan et al.	10	100	100	20	100
	15	60	100	0	60
	20	20	100	0	80
Zhu and Hedy	10	80	100	N/A	100
	15	40	100	N/A	20
	20	20	100	N/A	N/A

3. *Processing time to setup time ratio*: In the primary experimental study, the ratio of processing times to setup times was approximately 4 to 1. In this side experiment, ratios on both sides of this were used, one where the ratio was 1 to 1 and one where the ratio was 10 to 1. The processing and setup times were scaled to ensure objective function values similar to those of the primary experimental study and to focus solely on the effects of the different processing to setup time ratios on solution time. No changes in problem solution times were seen at the different ratios or between the different formulations.

4. Conclusions

This paper introduces a new formulation for sequence dependent scheduling that makes use of network flow ideas. Besides the intuitive value of this formulation, it also has computational advantages. Since the number of variables and constraints in the model do not grow with the number of machines, the new formulation performed much better than other formulations for larger multiple-machines problems. A shortcoming of the experimental results is the limited sized problems that can be solved. It would be nice to be able to solve problems with a larger number of jobs.

This research gives evidence that for sequence-dependent setup problems, the proposed network formulation performs quite well when many machines are involved. It would seem that the difficulty of the problem is some function of N/M for the new formulation rather than $N \times M$ as for the other two formulations. It also shows that the Balakrishnan et al. formulation is better for single machine problems and problems with a relatively small number of machines. The Zhu and Hedy formulations seem dominated by the other two formulations for the problems solved but showed relative improvement as the number of jobs increased.

Several extensions to the network problem formulation can be accomplished with minor changes or additions to the model. If desired for a more dynamic environment, constraints for release dates, r_i , could be added as shown in Eq. (34) below. This constraint essentially subtracts the processing and setup times from the completion time of a job in order to arrive at its starting point, and it ensures the start time of every job is greater than or equal to the release date.

$$B - \sum_{\substack{j=0 \\ i \neq j}}^N y_{ij} - p_i \geq r_i \quad \forall i = 1 \dots N. \quad (34)$$

Development of good intuitive heuristics for this problem would be a valuable extension as well since solving large problems optimally with any of the formulations tested would require very long solution times. Along with these heuristics, the development of good lower bounds on the optimal objective value might be helpful in pursuing a branch and bound approach to solving the problem optimally.

Another problem type this formulation could be well suited for is the multiple traveling salesman problem with time windows (TSPTW). Pesant et al. [10] show the TSPTW consists of finding a minimum cost tour of a set of cities where each city is visited exactly once. To be feasible, the tour must start and end at a unique depot within a certain time window and each city must be visited within its own time window. Thus this problem is one of finding sequences for each salesman just as the formulation presented finds sequences for each machine. The changeover times are equivalent to travel times and job processing time is the same as time spent at a site. The time window has parallels as well, with the job ready time being equivalent to the start of the time window and the due date being the same as the end of the time window. There are many practical applications for the TSPTW. Any distribution scheduling problem where clients are not open every day of the week or every hour of the day is a TSPTW. Christensen and Fagerholt [11] have shown ocean shipping is another example of where the TSPTW can be used, as ports are often closed on nights and weekends and the process of loading or unloading cargo often takes more than one day.

References

- [1] S.S. Panwalker, R.A. Dudek, M.L. Smith, Sequencing research and the industrial scheduling problem, in: S.E. Elmaghraby (Ed.), *Symposium on the theory of Scheduling and its Applications*, 1973, pp. 29–38.
- [2] K.R. Baker, *Introduction to Sequencing and Scheduling*, Wiley, New York, 1974.
- [3] Z. Zhu, R.B. Heady, Minimizing the sum of earliness/tardiness in multi-machine scheduling: a mixed integer programming approach, *Computers and Industrial Engineering* 38 (2000) 297–305.
- [4] N. Balakrishnan, J.J. Kanet, S.V. Sridharan, Early/tardy scheduling with sequence dependent setups on uniform parallel machines, *Computers and Operations Research* 26 (1999) 127–141.
- [5] B.J. Coleman, Technical note: a simple model for optimizing the single machine early/tardy problem with sequence-dependent setups, *Production and Operations Management* 1 (2) (1992) 225–228.
- [6] Z. Chen, Scheduling with batch setup times and earliness-tardiness penalties, *European Journal of Operational Research* 96 (1997) 518–537.
- [7] F. Sivrikaya-Serifoglu, G. Ulusoy, Parallel machine scheduling with earliness and tardiness penalties, *Computers and Operations Research* 26 (1999) 773–787.
- [8] X. Zhu, W.E. Wilhelm, Scheduling and lot sizing with sequence-dependent setup: a literature review, *IIE Transactions* 38 (2006) 987–1007.
- [9] CPLEX Interactive Optimizer 7.1.0 with Simplex, Mixed Integer & Barrier Optimizers; Copyright ILOG 1997–2001.
- [10] G. Pesant, M. Gendreau, J. Potvin, J. Rousseau, On the flexibility of constraint programming models: from single to multiple time windows for the traveling salesman problem, *European Journal of Operational Research* 117 (1999) 253–263.
- [11] M. Christiansen, K. Fagerholt, Robust ship scheduling with multiple time windows, *Naval Research Logistics* 49 (2002) 611–625.